

Classificação de Sinais Vitais utilizando Rede Neural MLP e Árvores de Decisão

Samuel Stroschein Yokoyama

Sistemas Inteligentes – UTFPR – Prof. Gustavo Alberto Gimenez Lugo

Resumo

Este documento detalha a modelagem, implementação e análise de algoritmos de aprendizagem de máquina para o problema de triagem e classificação de sinais vitais de vítimas de catástrofes. Foram implementadas duas abordagens, baseada em drvore de decisão, e uma, baseada em redes neurais. O estudo detalha as técnicas de pré-processamento aplicadas, como escalonamento min-max, a parametrização dos modelos, e comparação dos resultados obtidos. Por fim, reflete-se sobre os desdobramentos éticos da utilização destes sistemas em cenários reais.

1 Introdução

Em cenários de catástrofes, a triagem rápida de vítimas é um fator crítico para a sobrevivência. O corpo médico utiliza sinais vitais para classificar os pacientes em quatro categorias: crítico, instável, potencialmente estável e estável.

Este trabalho utiliza técnicas de Inteligência Artificial para reconstruir a função de mapeamento entre os sinais vitais brutos e as respectivas classes de gravidade. Para a implemenhtação, optou-se pela linguagem Java, utilizando-se a arquitetura na imagem.

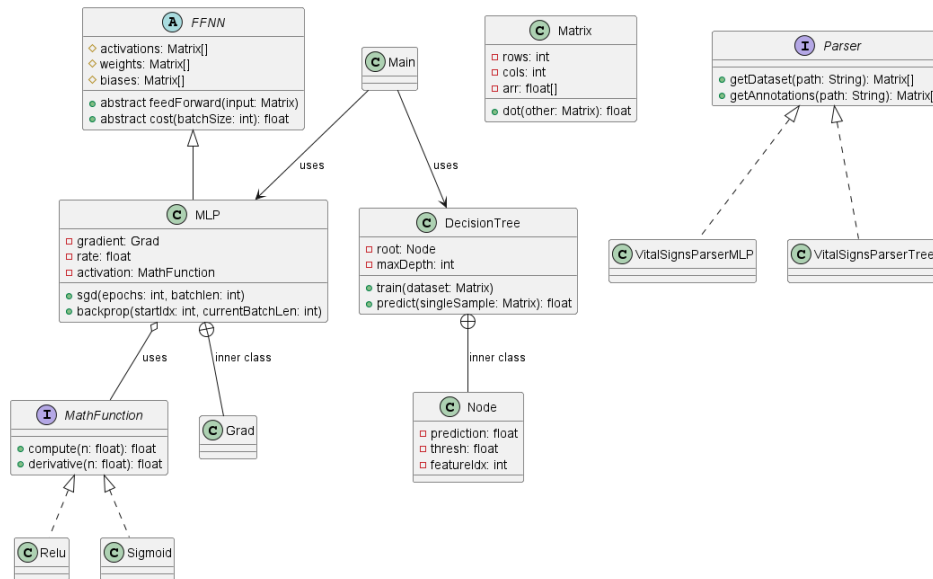


Figura 1: Diagrama de Classes da implementação gerado utilizando a extensão PlantUML no VSCode.

2 Requisitos e Modelagem

A modelagem do problema foi definida considerando o conjunto de dados fornecido, extraindo apenas as características clinicamente e matematicamente relevantes para a classificação.

2.1 Atributos de Entrada e Saída

Os dados históricos originais continham colunas redundantes ou informativas que foram descartadas na fase de modelagem para evitar vazamento de dados (*data leakage*). Os atributos de entrada selecionados foram estritamente os vetores si_3 (Qualidade da Pressão Arterial - qPA), si_4 (Pulsção) e si_5 (Frequência Respiratória).

O atributo de saída (y_1) corresponde à classe de gravidade, categorizada em um intervalo de 1 a 4.

2.2 Requisitos Não-Funcionais

Para ambas as modelagens, exigiu-se que o sistema rodasse inteiramente em CPU, implementado em Java. O sistema deve carregar os datasets através de **parsers**, que são a implementação da interface Parser, que preparam o dataset para o formato utilizado pelo MLP e DecisionTree.

3 Metodologia e Parametrização

3.1 Árvore de Decisão

Foi implementado um algoritmo do tipo CART (*Classification and Regression Trees*). A escolha do melhor nó de divisão foi guiada pelo cálculo da Impureza de Gini.

Para prevenir o superajuste (*overfitting*) sobre as instâncias de treinamento, parametrizou-se um limite de profundidade máxima da árvore ($maxDepth = 5$). O algoritmo realiza divisões recursivas no espaço de dados até atingir folhas puras, o limite de profundidade ou uma quantidade ínfima de amostras restantes no nó.

3.2 Rede Neural MLP (Aprendizagem Conexionista)

A topologia escolhida para a rede neural constitui-se de uma camada de entrada com 3 neurônios, uma camada oculta alargada com 16 neurônios e uma camada de saída com 4 neurônios, sendo cada uma representando uma das classes.

Analisando o dataset, foi possível observar que havia uma amplitude muito grande entre os valores, alguns próximos a 200, enquanto a qPA possuía valores negativos. Isso causava a saturação da função Sigmoid, que ficava sempre em 0 ou 1 para números com valor absoluto muito grande. Este efeito zerava a derivada durante Backpropagation [1] (*vanishing gradient*), impedindo o aprendizado. Para contornar este limite:

- **Normalização Min-Max:** Os dados de entrada foram normalizados. O **parser** foi projetado para salvar na memória os parâmetros (mínimos e amplitudes) encontrados no treino.
- **One-Hot Encoding:** A saída foi modelada para representar probabilidades independentes, mapeando as 4 classes para vetores binários discretos (ex: $[1, 0, 0, 0]$), penalizando a rede corretamente.
- **Random Restart:** Foi desenvolvido um ambiente automatizado dentro da classe Main, que reinicializa os pesos aleatórios e executa mini batches para fugir de mínimos locais, sempre armazenando quais os melhores valores para os pesos e vies. Depois, executa mais um fine tuning final (5000 épocas com $\alpha = 0.01$).

4 Resultados e Comparação

O treinamento foi realizado utilizando o mesmo conjunto de dados pré-rotulados, o desempenho do sistema foi medido com os dados históricos para validar a capacidade de convergência das implementações.

A **Árvore de Decisão** apresenta precisão maior e um tempo de treinamento quase nulo. No entanto, seu alto desempenho, pode ser indicativo de tendência a memorização.

A **Rede Neural MLP**, com um tempo muito maior de treinamento, e maior complexidade de parametrização (normalização), atua traçando hiperplanos contínuos no espaço tridimensional das *features*. Embora registre 70% de acurácia no cenário basal de treinamento.

Modelo	Acurácia Base (Treinamento)
Árvore de Decisão (CART)	~ 86.20%
Rede Neural (MLP)	~ 71.12%

Tabela 1: Resultados comparativos do desempenho dos modelos desenvolvidos.

5 Desdobramentos e Impacto Humano

A incorporação de modelos de Aprendizagem de Máquina em triagem médica suscita reflexões imperativas.

Explicabilidade e Confiança Médica: A modelagem simbólica (Árvores) apresenta uma enorme vantagem ética, pois seu fluxo de decisão pode ser lido e auditado por um médico humano ("Se o pulso exceder X e a respiração for Y , risco crítico"). A MLP, configurando-se como uma caixa-preta de pesos e vieses, gera barreiras à confiança clínica, exigindo sistemas de IA explicável para justificar seus resultados.

Viés Algorítmico e Justiça: O sistema de triagem depende do cenário de coleta. Se a base de treinamento reflete os padrões fisiológicos de jovens adultos, idosos ou crianças (cujos limiares vitais são distintos) podem ser sub-classificados, retardando o atendimento e prejudicando severamente grupos demográficos vulneráveis.

Por fim, quanto à **Autonomia**, a ferramenta, não se mostra confiável para situações reais, mas caso exista tal ferramenta no ambiente, a ferramenta deve agir unicamente como apoio secundário, mas delegando a responsabilidade de classificação para um humano com competência para julgar a que classe o indivíduo pertence.

6 Conclusão

O presente estudo demonstrou que a aplicação de tais técnicas de inteligência artificial, seguindo a implementação descrita, não se mostra confiável o suficiente para a automação de triagem em desastres. A implementação explicitou os gargalos algebricos de cada modelo e consolidou a compreensão de que não existe um modelo ótimo. Enquanto as árvores fornecem respostas altamente auditáveis e ágeis, as redes neurais oferecem uma resistência generalista a ruídos. Tais ferramentas não devem ser usadas em cenários reais, mas, no caso de serem, de ser feito o uso combinado e supervisionado de ambas abordagens.

7 Separação de Responsabilidades

- **Samuel Stroschein Yokoyama:** Implementação em Java, com o código-fonte disponível em repositório público [2], e elaboração deste relatório.

8 Tempo delegado

- **Implementação em java:** 15h00m
- **Elaboração deste documento:** 2h00m

Referências

- [1] Michael A. Nielsen. Neural networks and deep learning. Disponível em: <http://neuralnetworksanddeeplearning.com>, 2019. Acesso em: 21 mai. 2026.
- [2] Samuel Stroschein. Yokoyama. Código desenvolvido disponível em: <https://github.com/SmlYkm/Classificacao-Usando-Arvore-e-Redes-Neurais>. Acesso em: 22 mai. 2026.